

Tytuł: Sterownik serwomechanizmu z procesorem ucore

Autorzy: Krzysztof Muś, Oliwier Krupa

Ostatnia modyfikacja: 16.06.2026

Spis treści

1. Repozytorium git.....	1
2. Wstęp.....	1
3. Specyfikacja.....	2
3.1. Opis ogólny algorytmu	2
3.2. Tabela zdarzeń	2
4. Architektura.....	2
4.1. Moduł: top.....	2
4.1.1. Schemat blokowy	2
4.1.2. Porty	3
a) wejścia topu FPGA	3
b) wyjścia topu FPGA.....	3
4.1.3. Interfejsy	4
a) dbus / servo_if / uart_if.....	4
4.2. Rozprowadzenie sygnału zegara.....	4
5. Implementacja i weryfikacja	6
5.1. Lista zignorowanych ostrzeżeń Vivado	6
5.2. Wykorzystanie zasobów	6
5.3. Marginesy czasowe	6
6. Konfiguracja sprzętu	6

1. Repozytorium git

Adres repozytorium GITa:

https://github.com/krzysiek344/ucore-sevo_driver-integration

Repozytorium pomocnicze sterownika: <https://github.com/krzysiek344/servo-hw-driver>

Projekt był prowadzony w dwóch repozytoriach. Repozytorium servo-hw-driver zawiera pierwotne moduły RTL sterownika serwo, a repozytorium ucore-sevo_driver-integration zawiera wersję finalną: integrację z procesorem ucore, aplikację C, testy, top FPGA i skrypty do bitstreamu.

2. Wstęp

Projekt realizuje sprzętowo-programowy sterownik serwomechanizmu/napędu krokowego na płycie Basys3. Wcześniejszy sterownik RTL został włączony do gotowego projektu procesora ucore jako peryferium memory-mapped. Użytkownik wysyła komendy przez UART, program w języku C zapisuje rejestry peryferium, a logika RTL generuje fizyczną sekwencję faz i obsługuje czujnik pozycji zerowej.

3. Specyfikacja

3.1. Opis ogólny algorytmu

Po resecie BTNC top FPGA uruchamia PLL, synchronizuje reset i startuje SoC. Aplikacja C inicjalizuje UART oraz sterownik servo. Komendy callib, goto, pos i status, trafiają z terminala do programu C, a następnie przez DBUS do rejestrów servo.sv. Wrapper zamienia zapisy CPU na sygnały enable, callib, go_to, target_pos, scale_val oraz inversion. Moduł top_servo_drv.sv wykonuje sterowanie sprzętowe:

filtruje czujnik aktywny niskim poziomem, generuje step_tick, wybiera kierunek, aktualizuje licznik pozycji i wystawia fazy PO-0..PO-3.

Najważniejszy przepływ komunikacji to:

PC/terminal UART -> RsRx/RsTx -> program C na ucore -> DBUS -> servo.sv -> top_servo_drv.sv -> fizyczne piny PMOD. Czujnik servo jest podłączony jako servo_sensor_raw i jest aktywny stanem niskim.

3.2. Tabela zdarzeń

Tabela poniżej opisuje zdarzenia widoczne dla użytkownika oraz zdarzenia wewnętrzne, które powoduje program C i sterownik RTL.

Zdarzenie	Kategoria	Reakcja systemu
Naciśnięcie BTNC	Reset	Reset wejścia IO, synchronizacja resetu po PLL i wyzerowanie rejestrów SoC.
Start programu C	Inicjalizacja	UART i sterownik servo są konfigurowane, a sterownik zostaje włączony.
Komenda callib	Kalibracja	CR[1] uruchamia FSM w trybie szukania czujnika zerowego.
Czujnik raw = 0	Kalibracja	Po debouncingu FSM kończy bazowanie i zeruje licznik pozycji.
Komenda goto <N>	Ruch	Program wpisuje TARGET_POS i ustawia CR[2], a FSM wykonuje kroki do celu.
Komenda pos?	Diagnostyka	Program odczytuje CURRENT_POS i odsyła pozycje przez UART.
Komenda status?	Diagnostyka	Program odczytuje busy z SR i odsyła STATUS BUSY albo STATUS IDLE.
Nieznana komenda	Obsługa błędów	Program odsyła ERR CMD albo ERR ARG dla złego argumentu goto.

4. Architektura

Architektura jest hierarchiczna. Warstwa FPGA zawiera top Basys3, PLL i synchronizator resetu. Warstwa SoC zawiera rdzeń ucore, pamięci, UART, GPIO, timer oraz peryferium servo. Warstwa servo_driver zawiera właściwe bloki sterowania mechaniką.

Dla procesora sterownik jest zwykłym peryferium pod adresem bazowym 0x50000000. Dekodowanie adresu wykonuje dbus_arbiter.sv, natomiast servo.sv obsługuje lokalne offsety rejestrów.

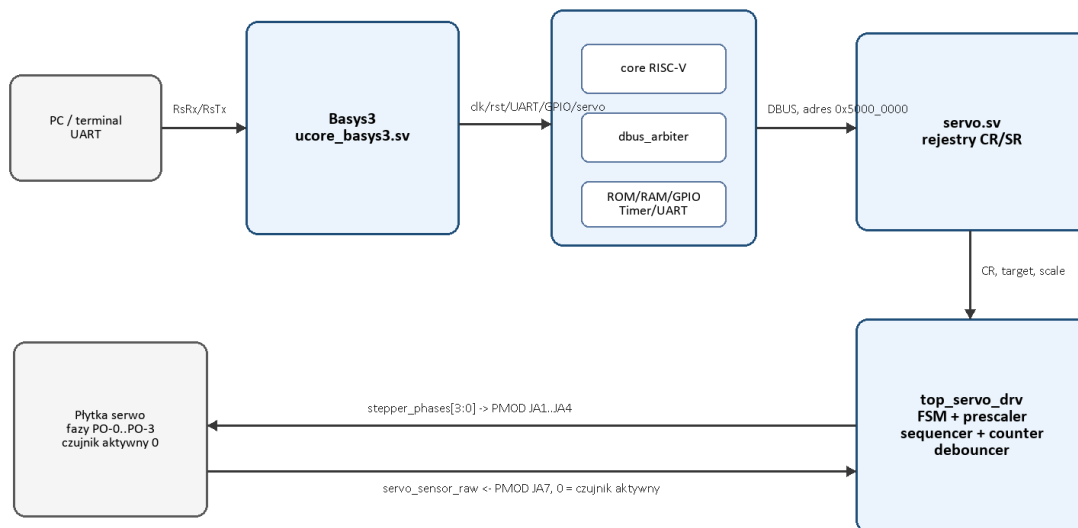
4.1. Moduł: top

Osoba odpowiedzialna: KM/OK

4.1.1. Schemat blokowy

Schemat pokazuje przepływ informacji od terminala UART do fizycznych pinów PMOD. Nie jest to schemat z Vivado, tylko logiczny podział odpowiedzialności między top FPGA, SoC, wrapper servo i sterownik mechaniki.

Architektura projektu: ucore + sterownik servo



Sterownik jest widoczny dla procesora jako peryferium memory-mapped. Program C wysyła komendy przez rejestry, a fizyczne wyjścia są wystawione na PMOD.

Rys. 1. Architektura projektu: terminal UART, top Basys3, SoC ucore, peryferium servo.v i sterownik top_servo_drv.v.

Opis bloków:

- ucore_basys3.v łączy fizyczne piny Basys3 z SoC, generuje zegar i reset.
- soc.v instancjuje rdzeń ucore, pamięci, UART, GPIO, timer oraz servo.
- servo.v udostępnia rejestry CPU, a top_servo_drv.v wykonuje sterowanie fizycznym mechanizmem.

4.1.2. Porty

a) *refclk, btnC, RsRx, sw[3:0], servo_sensor_raw - wejścia topu FPGA*

nazwa portu	opis
refclk	zegar referencyjny Basys3, wejście PLL
btnC	przycisk resetu układu
RsRx	wejście UART z terminala
sw[3:0]	przełączniki diagnostyczne doprowadzone do GPIO
servo_sensor_raw	surowy sygnał czujnika, aktywny stanem niskim

b) *RsTx, led[8:0], stepper_phases[3:0] - wyjścia topu FPGA*

nazwa portu	opis
RsTx	wyjście UART do terminala
led[8:0]	diody diagnostyczne stanu przełączników i faz
stepper_phases[3:0]	cztery fazy sterujące driverem serwo na PMOD JA1-JA4

4.1.3. Interfejsy**a) *dbus - magistrala danych do ROM/RAM/GPIO/timer/UART/servo***

nazwa sygnału	opis
dbus	magistrala danych rdzenia ucore do peryferiów SoC
servo_if	połączenie peryferium servo z fizycznym sterownikiem: fazy i czujnik
uart_if	komunikacja tekstowa z użytkownikiem przez RsRx/RsTx
memory_map	adres bazowy SERVO_BASE_ADDR = 0x50000000 wspólny dla RTL i C

4.1.4. Rejestry peryferium servo

Wrapper servo.sv udostępnia 5 rejestrów. Program C korzysta z tych samych offsetów w bibliotece sw/libs/soc/src/servo.c.

Rejestr / offset	Dostęp CPU	Znaczenie
CR / 0x000	R/W	Rejestr sterowania: enable, callib, go_to oraz inversion.
SR / 0x004	R	Status: callib_done, go_to_done, busy oraz sensor_raw. SR[3] pokazuje surowy stan pinu czujnika.
TARGET_POS / 0x008	R/W	Pozycja docelowa dla komendy goto.
CURRENT_POS / 0x00C	R	Aktualna pozycja zwracana przez licznik kroków.
SCALE / 0x010	R/W	Preskaler ruchu, czyli odstęp między kolejnymi krokami.

4.2. Rozprowadzenie sygnału zegara

Osoba odpowiedzialna: KM/OK

Źródłem zegara jest refclk płytki Basys3. Moduł pll.sv generuje zegar systemowy dla SoC, a reset_synchronizer.sv synchronizuje reset po sygnale pll_locked. Pozostałe moduły korzystają z jednego zegara clk.

Generator zegara znajduje się w module ucore_basys3.sv. Przycisk BTNC tworzy wejściowy reset io_rst_n, który jest dalej synchronizowany przed podaniem do SoC.

Projekt nie używa wielu niezależnych domen zegarowych. Komunikacja między modułami odbywa się w jednej domenie clk.

4.2.1. Moduły sterownika serwo

Sterownik mechaniki jest zbudowany z małych bloków RTL w katalogu hw/rtl/soc/servo_driver. Dzięki temu osobno testowano prescaler, licznik pozycji, sekwencer, debouncer, FSM oraz top driver.

Moduł	Plik	Rola w sterowniku
master_fsm	master_fsm.sv	Główna maszyna stanów: kalibracja, ruch do pozycji oraz flagi busy/done.
prescaler	prescaler.sv	Dzieli zegar i generuje impuls step_tick zgodnie ze scale_val.
sequencer	sequencer.sv	Zamienia step_tick i kierunek na sekwencję czterech faz silnika.
step_counter	step_counter.sv	Zlicza pozycje i zeruje licznik po znalezieniu czujnika.
debouncer	debouncer.sv	Stabilizuje sygnał czujnika i odrzuca krótkie zakłócenia.
top_servo_drv	top_servo_drv.sv	Łączy bloki sterownika i odwraca sensor active-low do logiki wewnętrznej.

5. Implementacja i weryfikacja

5.1. Po finalnym generowaniu bitstreamu Vivado 2025.1 zakończył pracę z wynikiem: 0 errors, 0 critical warnings, 21 warnings. Ostrzeżenia są opisane w tabeli poniżej. Warningi Synth 8-7129 dotyczą nieużywanych bitów magistrali dbus i mogą być świadomie odfiltrowane z warning_summary.log, natomiast ostrzeżenia implementacyjne pozostają widoczne i opisane.

ID	Wynik	Uzasadnienie
Synth 8-7129	filtr / opis	Dotyczy nieużywanych bitów dbus.addr oraz dbus.be. Dekodowanie adresu odbywa się w dbus_arbiter.sv, a peryferia używają lokalnych offsetów.
DRC REQ-1839 / CHECK-3	21	Wynika z BRAM code_rom sterowanej sygnałami zależnymi od rejestrów z resetem asynchronicznym w dostarczonym projekcie ucore. Bitstream powstaje, brak error i critical warning.
Power 33-332 / Synth 8-7080	info	Dotyczy estymacji mocy resetu oraz kryteriów syntezy równoległej. Nie jest błędem funkcjonalnym projektu.

5.2. Wykorzystanie zasobów

Bitstream generuje się poprawnie i został zapisany w results/ucore_basys3.bit. Skrypty w katalogu tools tworzą również warning_summary.log w katalogu results.

5.3. Marginesy czasowe

Raport timing summary po implementacji Vivado pokazuje dodatnie marginesy czasowe. WNS = 3.862 ns, WHS = 0.078 ns, TNS = 0.000 ns oraz THS = 0.000 ns. Liczba niespełnionych endpointów dla setup i hold wynosi 0, dlatego wymagania czasowe projektu są spełnione.

Parametr	Wartość	Wniosek
WNS	3.862 ns	Dodatni margines setup.
TNS	0.000 ns	Brak sumarycznego naruszenia setup.
WHS	0.078 ns	Dodatni margines hold.
THS	0.000 ns	Brak sumarycznego naruszenia hold.
Failing endpoints	0	Brak ścieżek niespełniających timing.

6. Konfiguracja sprzętu

Nie dotyczy trybu multiplayer; projekt jest sterownikiem serwomechanizmu zaakceptowanym jako projekt sprzętowy.

Piny: stepper_phases [0..3] -> PMOD JA1-JA4, servo_sensor_raw -> PMOD JA7, UART przez USB-RS232.

Czujnik serwo jest aktywny stanem niskim. Wejścia/wyjścia PMOD pracują w standardzie LVCMOS33.